CDC Streaming Data Engineering Pipeline

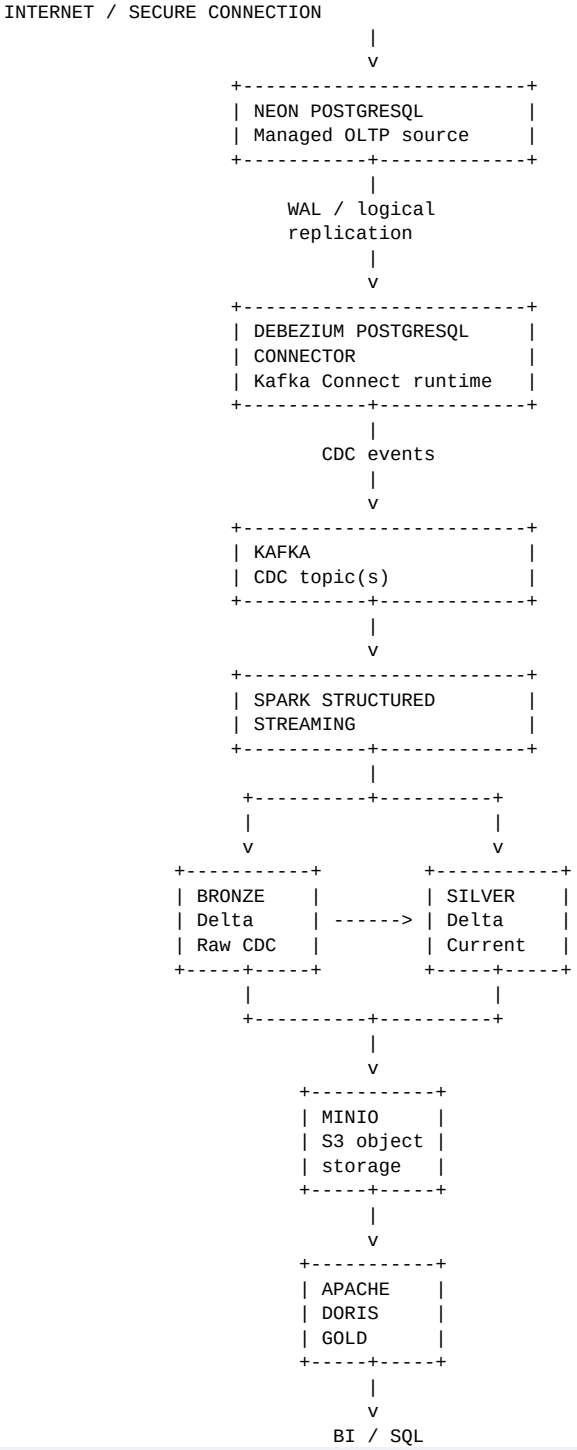
Neon PostgreSQL → Debezium → Kafka → Spark Structured Streaming → MinIO/Delta Lake → Apache Doris

Source decision: Neon PostgreSQL is used from the beginning. There is no local PostgreSQL database and no PostgreSQL Docker container in this project.

Purpose: This document is the implementation specification for building a realistic end-to-end CDC streaming data engineering project and can be supplied to AI coding assistants as project context.

Version 2.0 • August 2026

1. Architecture



Responsibilities: Neon = source of truth; Debezium = CDC capture; Kafka = event transport/replay; Spark = streaming processing; Bronze = raw CDC history; Silver = current state; MinIO = object storage; Delta Lake = table format; Doris = analytical serving.

2. Core Project Goal

A Python application connects directly to Neon, seeds a 10-column table, and continuously performs INSERT, UPDATE, DELETE, batch operations, and transactions. Neon/PostgreSQL records committed changes in WAL. Debezium reads those changes through logical replication and publishes CDC events to Kafka. Spark Structured Streaming consumes Kafka, writes raw events to Bronze Delta, reconstructs current state in Silver, and produces Gold business aggregates for Apache Doris.

Important: Debezium does not consume Python/application log files. The Python program creates database changes; Debezium captures those database changes.

3. Neon Source Database

Create the source database in Neon before any local streaming infrastructure is started. The Python simulator and Debezium connector both connect to Neon remotely. The project therefore begins with a real managed PostgreSQL source instead of a local database.

Required Neon information: host, port, database, user, password/secret, SSL requirements, and confirmation that the Neon project/plan supports the logical replication capability required by the chosen Debezium approach.

Do not assume self-managed PostgreSQL settings are available on Neon. Neon is managed infrastructure. Current Neon documentation must be checked when configuring logical replication, permissions, replication slots, publications, or related settings.

4. Source Table — 10 Columns

Use one deliberately simple table to make CDC behavior easy to understand and test.

Column	Type	Purpose
transaction_id	BIGSERIAL PK	CDC key
customer_id	INTEGER	Customer identifier
customer_name	VARCHAR	Customer name
product_id	INTEGER	Product identifier
product_name	VARCHAR	Product name
quantity	INTEGER	Units
unit_price	DECIMAL(10,2)	Unit price
total_amount	DECIMAL(12,2)	Transaction value
status	VARCHAR	Business state
updated_at	TIMESTAMP	Last update time

```
CREATE TABLE customer_transactions (
    transaction_id BIGSERIAL PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    customer_name VARCHAR(100),
    product_id INTEGER,
    product_name VARCHAR(150),
    quantity INTEGER,
    unit_price DECIMAL(10,2),
    total_amount DECIMAL(12,2),
    status VARCHAR(30),
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5. Python Neon Data Simulator

The simulator uses psycopg or another PostgreSQL driver and connects directly to Neon. Faker/random generates realistic values. All connection details are supplied through environment variables.

```
NEON_DATABASE_URL=postgresql://<user>:<password>@<neon-host>/<database>?sslmode=require
```

Required operations: single INSERT; batch INSERT; single UPDATE; bulk UPDATE; single DELETE; batch DELETE; multi-operation COMMIT; ROLLBACK; controlled database errors. The simulator should support configurable seed size and operation frequency.

The application logs what it does for observability, but Debezium does not read those log files.

6. Neon CDC / Logical Replication Prerequisites

Before Debezium is configured, verify the logical replication mechanism available for the Neon project. The source-side requirements generally include appropriate logical decoding/WAL capability, a usable replication slot, publication/table configuration, suitable privileges, and network/TLS connectivity.

The exact Neon procedure is intentionally not hard-coded here because managed-service capabilities and setup procedures can change. During implementation, use the current Neon documentation and verify the selected plan supports the required CDC workflow.

7. Debezium + Kafka Connect

Debezium runs as a PostgreSQL source connector inside Kafka Connect. Kafka Connect is the runtime; Debezium provides the CDC connector.

```
Neon PostgreSQL
      |
      | logical replication
      v
Debezium PostgreSQL Connector
      |
      | Kafka Connect
      v
Kafka
```

```
{
  "name": "neon-postgres-cdc",
  "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
  "database.hostname": "<NEON_HOST>",
  "database.port": "5432",
  "database.user": "<CDC_USER>",
  "database.password": "<CDC_PASSWORD>",
  "database.dbname": "<NEON_DATABASE>",
  "topic.prefix": "neon_cdc",
  "plugin.name": "pgoutput",
  "slot.name": "neon_cdc_slot"
}
```

This is a configuration template. Validate exact properties against the selected Debezium version and current Neon requirements.

8. Kafka CDC Topic

With topic prefix neon_cdc, a typical table topic is expected to resemble:

```
neon_cdc.public.customer_transactions
```

Debezium/Kafka Connect is the producer of database CDC events. A separate Python Kafka producer is not required for this path.

Preserve topic, partition, offset, event timestamp, operation, and source metadata where available.

9. Debezium Event Semantics

CDC events contain before/after row state and metadata. Exact serialization depends on the selected Kafka converter.

```
{
  "before": {"transaction_id": 1001, "status": "PLACED"},
  "after":  {"transaction_id": 1001, "status": "SHIPPED"},
  "op": "u",
  "source": {"lsn": "...", "table": "customer_transactions"}
}
```

c = create/insert; u = update; d = delete; r = snapshot/read.

DELETE handling must be explicitly implemented because the current row may not exist in after. Snapshot/read events must also be distinguished from live inserts.

10. Spark Structured Streaming

Spark reads the Debezium Kafka topic rather than polling Neon. It decodes the Kafka value, parses the CDC envelope, preserves metadata, and writes Bronze/Silver.

```
Kafka
  |
  v
Spark readStream
  |
  v
```

```
Decode / parse Debezium
  |
  +----> Bronze Delta
  |
  +----> Silver CDC processing
```

Use durable Spark checkpoints in persistent object storage. Do not rely on an ephemeral container filesystem for production-style recovery.

11. Bronze Layer

Bronze is the raw CDC history layer. It must preserve enough information to understand exactly what Spark received and to trace downstream results backward.

Recommended fields: primary key; before payload; after payload; operation; event timestamp; source table; source LSN/transaction metadata; Kafka topic; partition; offset; ingestion timestamp.

```
s3a://cdc-lake/bronze/customer_transactions/
```

DELETE events remain in Bronze even though the row disappears from Silver.

12. Silver Layer

Silver reconstructs the current state of customer_transactions from CDC events.

```
Events:
INSERT 101
UPDATE 101
UPDATE 101
DELETE 101

Final Silver:
transaction_id=101 -> absent
```

Rules: INSERT creates; UPDATE merges/replaces by primary key; DELETE removes current state; snapshot/read initializes state; retries/replays must not corrupt final state.

A practical Spark + Delta implementation can use foreachBatch and deterministic Delta MERGE/upsert/delete logic. Exact APIs must match selected Spark/Delta versions.

13. MinIO + Delta Lake

MinIO is the S3-compatible object store. Delta Lake is the table format. Spark is the processing engine writing the tables.

```
MinIO
|
+-- bronze/customer_transactions/
+-- silver/customer_transactions/
+-- checkpoints/bronze/
+-- checkpoints/silver/
```

MinIO answers where files live; Delta Lake answers how those files behave as tables.

14. Apache Doris Gold

Doris is the analytical serving layer. Gold datasets should be business-oriented rather than raw CDC envelopes.

Candidate tables: gold_transaction_summary; gold_customer_metrics; gold_product_metrics; gold_realtime_metrics.

```
SELECT
    status,
    COUNT(*) AS transaction_count,
    SUM(total_amount) AS total_revenue,
    AVG(total_amount) AS avg_transaction_value
FROM silver_customer_transactions
GROUP BY status;
```

15. Repository Structure

```
data-engineering-cdc-pipeline/
|
```

```
|-- README.md
|-- docker-compose.yml
|-- .env.example
|-- .gitignore
|
|-- database/
|   `-- neon/
|       `-- 01_create_schema.sql
|
|-- simulator/
|   `-- seed_database.py
|
|-- debezium/
|   |-- connector-config.json
|   `-- register_connector.sh
|
|-- kafka/
|   `-- config/
|
|-- streaming/
|   |-- spark_session.py
|   |-- bronze/bronze_cdc.py
|   |-- silver/silver_cdc.py
|   `-- utils/
|
|-- storage/
|   `-- minio/README.md
|
|-- doris/
|   |-- ddl/
|   `-- load/
|
|-- scripts/
|   |-- start.sh
|   |-- stop.sh
|   `-- health_check.sh
|
`-- docs/
    |-- architecture.md
    |-- neon_setup.md
    |-- data_contract.md
    |-- testing.md
    |-- troubleshooting.md
    `-- operations.md
```

No PostgreSQL Docker service appears in this structure. Docker may be used for Kafka, Kafka Connect/Debezium, MinIO, Spark, and Doris, but Neon remains the database source.

16. Infrastructure Boundary

The project is intentionally hybrid: Neon is managed cloud infrastructure; the streaming/lake/serving stack can be local or hosted.

Potential local services: Kafka, Kafka Connect + Debezium, Kafka UI, MinIO, Spark runtime, Apache Doris. PostgreSQL is not included.

17. Environment Variables

```
# Neon
NEON_DATABASE_URL=
NEON_HOST=
NEON_PORT=5432
NEON_DATABASE=
NEON_USER=
NEON_PASSWORD=
NEON_SSLMODE=require

# Kafka
KAFKA_BOOTSTRAP_SERVERS=
KAFKA_CDC_TOPIC=

# MinIO
MINIO_ENDPOINT=
MINIO_ACCESS_KEY=
MINIO_SECRET_KEY=
MINIO_BUCKET=

# Doris
```

```
DORIS_HOST=
DORIS_PORT=
DORIS_DATABASE=
DORIS_USER=
DORIS_PASSWORD=
```

Never commit the Neon connection string, password, MinIO keys, or Doris credentials.

18. Networking

Both the Python simulator and the Kafka Connect/Debezium runtime must be able to reach Neon. Test Neon connectivity independently before debugging CDC.

Verify DNS resolution, outbound network access, port 5432 access where applicable, TLS/SSL configuration, and the authentication method. A connector container that cannot reach Neon cannot create a CDC stream.

19. CDC Correctness Requirements

The pipeline must satisfy the following:

1. Only committed source changes become valid downstream state.
2. Primary keys are preserved.
3. Per-key ordering is respected.
4. DELETEs remain visible in Bronze.
5. Silver is safe under retries/restarts.
6. Kafka offsets and Spark checkpoints are durable.
7. Snapshot events are distinguished from live events.
8. Malformed events are explicitly handled.
9. Gold reconciles with Silver.

20. Testing Plan

Test at every layer, not only by checking whether data eventually appears in Doris.

```
INSERT        -> Bronze row + Silver row
UPDATE        -> before/after in Bronze + new Silver state
DELETE        -> delete in Bronze + row absent from Silver
COMMIT        -> committed changes captured
ROLLBACK      -> rolled-back changes absent from final state
MULTI-UPDATE  -> final Silver equals source
RESTART       -> Spark resumes correctly
REPLAY        -> final Silver remains correct
NETWORK FAIL  -> connector recovery is verified
GOLD CHECK    -> Doris aggregate equals independent calculation
```

21. Observability / Debugging

```
Python simulator
      ↓
Neon PostgreSQL
      ↓
Logical replication / slot
      ↓
Debezium connector
      ↓
Kafka topic / partition / offset
      ↓
Spark query / checkpoint
      ↓
Bronze Delta
      ↓
Silver Delta
      ↓
Doris Gold
```

When a result is wrong, trace a primary key through the pipeline from left to right. Do not change Spark code until you have confirmed the expected CDC event exists in Kafka.

22. Failure and Recovery Model

Neon outage affects source connectivity. Debezium outage affects CDC capture. Kafka outage affects event transport. Spark outage affects downstream processing. Doris outage affects serving but should not destroy lake data. Durable Kafka retention, replication-slot behavior, Spark checkpoints, and Bronze history form the recovery strategy.

Operational warning: an inactive logical replication consumer can cause source-side WAL/storage pressure. Monitor the replication slot and Neon storage/health metrics.

23. Security

Use a dedicated CDC account where appropriate, minimum privileges, TLS/SSL for Neon, environment variables/secret management, private infrastructure endpoints, and sanitized logs. Never expose credentials in connector JSON committed to Git.

24. Implementation Roadmap

Phase 1 — Neon project and secure connectivity.
Phase 2 — Create the 10-column table in Neon.
Phase 3 — Build Python simulator against Neon.
Phase 4 — Verify Neon logical replication/CDC prerequisites.
Phase 5 — Run Debezium/Kafka Connect against Neon.
Phase 6 — Verify INSERT/UPDATE/DELETE events in Kafka.
Phase 7 — Build Spark Bronze on MinIO/Delta.
Phase 8 — Build Silver current-state processing.
Phase 9 — Build Doris Gold aggregates.
Phase 10 — Reliability, replay, restart, failure, reconciliation tests.
Phase 11 — Documentation and cleanup.

25. Recommended Build Discipline

1. Prove Python → Neon connectivity.
2. Prove Neon table CRUD.
3. Prove one INSERT → Debezium → Kafka event.
4. Prove UPDATE → Kafka event.
5. Prove DELETE → Kafka event.
6. Only then build Spark Bronze.
7. Then Silver.
8. Then Doris Gold.
9. Finally test failure/replay/restart.

This avoids debugging multiple independent systems simultaneously.

26. AI-Assisted Development Prompt

```
We are building the Neon-based CDC Streaming Data Engineering Pipeline.

Architecture:
Neon PostgreSQL -> Debezium/Kafka Connect -> Kafka ->
Spark Structured Streaming -> MinIO/Delta Lake -> Apache Doris.

Critical constraint:
There is NO local PostgreSQL Docker container.
Neon is the source database from the beginning.

Current phase: <PHASE>
Current component: <COMPONENT>

Requirements:
1. Follow this architecture.
2. Verify current Neon CDC/logical-replication requirements.
3. Do not assume self-managed PostgreSQL settings exist on Neon.
4. Use environment variables for credentials.
5. Give complete file paths and code.
6. Give exact run commands.
7. Give verification queries/commands.
8. Explain expected behavior.
```

```
9. Include failure modes and debugging.
10. Preserve CDC metadata needed for traceability/replay.
11. Implement only the requested phase.
12. Do not claim success without verification.
```

For each implementation ask the AI for: code → explanation → run commands → verification → expected result → troubleshooting → tests.

27. Definition of Done

Neon is the source; no local PostgreSQL is required; the Python simulator continuously changes Neon; Debezium captures changes; Kafka transports events; Spark consumes them; Bronze and Silver Delta tables live in MinIO; Silver correctly applies INSERT/UPDATE/DELETE; checkpoints support restart; Doris serves Gold aggregates; reconciliation tests pass; credentials are protected; setup and troubleshooting are documented.

28. Final Architecture Principle

Neon is the source of truth. Debezium captures change. Kafka transports change. Spark processes change. Bronze preserves change history. Silver reconstructs state. MinIO stores the lakehouse. Delta Lake provides table semantics. Doris serves business aggregates.

Every Gold metric should be traceable backward through Silver, Bronze, Kafka, Debezium metadata, and ultimately to a change made in Neon.

End of Project Specification